A Hybrid Classical–Transformer System for Hate Speech and Target Category Identification in Kannada–English Code-Mixed Text

Anonymous ACL submission

Abstract

This paper describes our submission to the HASTIKA shared task at ICON 2026, which covers hate speech detection (Task A) and hate target classification (Task B) in Kannada–English code-mixed comments. Our system pairs a stacked ensemble of linear and Naive Bayes classifiers, built on word and character TF-IDF features, with XLM-R and MuRIL encoders fine-tuned with R-Drop and label smoothing. All models were compared on one five-fold split of the training data and on the official development set. For Task A, a weighted average of the three models reached a development macro-F1 of 0.8288 and an official test macro-F1 of 0.7977 (rank 18 of 30). For Task B, both encoders scored well below the classical stack on the development set, and MuRIL never predicted two of the six classes on the development set, so we submitted the classical model alone; it reached a test macro-F1 of 0.6246 (rank 15 of 29). On data of this size, sparse character-aware features were hard to beat, and the encoders added value only on the balanced binary task.

1 Introduction

Kannada speakers often mix Kannada and English in online comments, and they usually type Kannada in Latin letters rather than in its own script. In the HASTIKA data nearly every comment is in Latin script, and the same Kannada word can be spelled in several ways. For hate speech detection this creates three practical problems: spelling variation, a small annotated corpus, and noisy raw text (emojis, stretched letters and, in our data, damaged character encodings).

The HASTIKA shared task at ICON 2026 is built on the HASTIKA corpus (Kavatagi and Rachh, 2025) and has two subtasks. In Task A, a comment is labelled HATE or NON-HATE. In Task B, a comment that is already known to be hateful is assigned one of six target categories: Gender, Geopolitical, Others, Political, Religion or Violence. Task B is also strongly imbalanced; the training set holds 1,362 Gender comments but only 186 Geopolitical ones. Systems are ranked by macro-F1.

Our system uses two kinds of model side by side: a stack of linear and Naive Bayes classifiers over sparse n-gram features, and two pretrained multilingual encoders. Both are evaluated on the same stratified five-fold split of the training data and on the same development set, so their strengths on each subtask can be compared directly. Our contributions are:

- a cleaning step that repairs text whose UTF-8 bytes were read as Latin-1 (mostly emojis, but also all Kannada-script text in the data), and a classical stack whose vectorizers are fitted inside every fold;
- XLM-R and MuRIL classifiers trained with R-Drop and label smoothing under five-fold cross-validation;
- a subtask-specific choice of final model: a three-model probability average for Task A and the classical stack alone for Task B, with the evidence for each choice.

2 Related Work

Hate speech detection. Early hate speech detectors treated the problem as text classification over lexical features, and word and character n-grams combined with a linear classifier remained a strong baseline for years (Davidson et al., 2017). Surveys point out that definitions, annotation schemes and evaluation practice differ widely between datasets, which makes systems hard to compare (Schmidt and Wiegand, 2017; Fortuna and Nunes, 2018). Shared tasks such as HatEval (Basile et al., 2019) and OffensEval (Zampieri et al., 2019) helped move the field towards fine-tuned pretrained encoders, and encoders adapted to abusive language

have since been proposed (Caselli et al., 2021). Some resources go beyond a binary decision and also annotate who is targeted. HateXplain, for example, labels target communities and word-level rationales (Mathew et al., 2021), and OffensEval includes a target subtask. Hate target identification is the setting of our Task B.

Code-mixed and Indian-language data. Social media text from India is frequently code-mixed and romanized. Bohra et al. (2018) released a Hindi–English code-mixed dataset for hate speech detection. For the Dravidian languages, Dravidian-CodeMix (Chakravarthi et al., 2022) provides sentiment and offensive language labels for code-mixed text, and the shared task on offensive language identification in Tamil, Malayalam and Kannada (Chakravarthi et al., 2021) brought together many systems on these data. For Kannada–English in particular, KanCMD (Hande et al., 2020) provides code-mixed comments labelled for sentiment and offensive language, and HASTIKA (Kavatagi and Rachh, 2025) annotates hate speech together with its target, which makes it suitable for the two-level task studied here.

Multilingual encoders and code-mixing. Multilingual pretrained encoders such as mBERT (Pires et al., 2019) and XLM-R (Conneau et al., 2020) transfer well across many languages. XLM-R is pretrained on text in about one hundred languages. MuRIL (Khanuja et al., 2021) and IndicBERT (Kakwani et al., 2020) are pretrained specifically on Indian languages, and MuRIL also includes transliterated text, which makes it a natural candidate for Latin-script Kannada. Benchmarks and studies on code-switching, such as GLUECoS (Khanuja et al., 2020) and the analysis of Winata et al. (2021), show that code-mixed input remains a harder case than monolingual text. Romanized text is an additional difficulty, because much of the pretraining data for these languages is written in the native script, and the subword vocabulary may split romanized words into many fragments.

Sparse features versus encoders. Simple n-gram models are often stronger than expected. Wang and Manning (2012) showed that Naive Bayes features combined with a linear model are a competitive baseline on small text classification datasets, and Complement Naive Bayes (Rennie et al., 2003) was designed for the skewed class distributions that small datasets often have. Character n-grams do not depend on a fixed spelling, which is attractive for romanized text. Whether a pretrained encoder beats such a well-tuned sparse baseline on a few thousand labelled comments is an empirical question, and one that this task lets us test.

Imbalance, regularization and ensembling. Two regularizers matter for our fine-tuning setup. R-Drop (Liang et al., 2021) adds a penalty on the disagreement between two dropout-perturbed predictions of the same input, and label smoothing (Szegedy et al., 2016) softens the training targets. Class imbalance is usually handled by class weighting, oversampling such as SMOTE (Chawla et al., 2002), or objectives such as focal loss (Lin et al., 2017). We use only class weighting, and we return to the other options as future work. For the classical side we use stacked generalization (Wolpert, 1992), in which a meta-learner combines the predictions of several base models.

Our work differs from most shared-task reports in that we evaluate a sparse stack and two encoders under the same folds, the same development set and the same cleaning, and we report per-class results, so that the cases in which each model family fails can be seen directly.

3 Task and Data

We used the official partitions without change (Table 1). Task A is close to balanced. Task B contains only hateful comments and is skewed towards Gender; Violence and Geo-political are the smallest classes in both training and development data. We checked the comment IDs of the train, development and test files of each task and found no overlap. Development data were used for model comparison and for choosing ensemble weights; test labels were never used.

The comments are short and almost entirely in Latin script. In the Task A training set the mean length is 11.5 words (median 8, 95th percentile 31), and 99.3% of the comments (6,398 of 6,446) contain no Kannada script at all; the other 45 mix Kannada script with Latin letters. About 15% of the comments (987) show the encoding damage described in Section 4.1, and 936 (14.5%) contain emojis once that damage is repaired. The Task B training set looks similar (mean 12.9 words, 4 comments with Kannada script). Almost all of the text the models see is therefore Latin-script, and any Kannada in it is romanized.

	Train	Dev	Test
Task A	6,446	806	806
Hate	3,160	395	–
Non-Hate	3,286	411	–
Task B	3,159	395	396
Gender	1,362	170	–
Political	559	70	–
Others	449	56	–
Religion	382	48	–
Violence	221	28	–
Geo-political	186	23	–

Table 1: Number of comments per partition, with class counts for the training and development data.

4 System Description

Figure 1 gives an overview. All models see the same cleaned text.

4.1 Text cleaning

In the released files, some non-ASCII characters were stored as UTF-8 bytes but read as Latin-1, so they appear as strings of unrelated symbols. For each run of non-ASCII characters we re-encode the characters as bytes and try to decode them as UTF-8; if decoding fails, the run is left as it was. In the Task A training set this changes 987 of 6,446 comments. Within those 987 changed comments, 934 contain emojis after the repair, 45 contain Kannada script (no comment in this partition contains Kannada script before the repair), and 28 contain other recovered symbols; a single comment may belong to more than one of these groups, since a UTF-8 run may mix an emoji with a Kannada letter or with other symbols. Across the whole training partition, the number of comments containing a valid emoji rises from 82 before the repair to 936 after. Without the repair, the emojis in particular would reach the models as noise. We then apply the following steps in this order: emojis are replaced by their English names (with the `emoji` package), HTML entities are unescaped and tags removed, URLs are deleted, user mentions become `@user`, any character repeated three or more times is cut to two, whitespace is collapsed and the text is lowercased. We did not try to transliterate or normalize Kannada spellings, so spelling variants reach the models unchanged.

4.2 Features

The classical branch uses two TF-IDF matrices with sublinear term frequency and a minimum document frequency of 2: word n-grams of length 1–3 (at most 30,000 features) and character n-grams of length 2–6 taken inside word boundaries (at most 50,000 features). Eight numeric features are appended and Min–Max scaled: comment length, uppercase ratio, number of "!" and "?", number of emoji tags, number of triple-character repeats, mean word length, number of tokens found in a hand-made list of 42 abusive and violence-related words (English and romanized Hindi), and digit ratio. Three of these features turned out to be almost inactive. The text is lowercased and repeats are shortened before the features are computed, and the emoji count looks for colon-delimited tags although emojis were converted to plain words. In the Task A training data, the uppercase ratio is non-zero for 3 of 6,446 comments, the repeat count for 15, and the emoji count for none; the word-list count is non-zero for 124. Vectorizers and scaler are fitted only on the training part of each fold, and on the full training set for the final model.

4.3 Classical stack

The base learners are a linear SVM (scikit-learn's `LinearSVC`, built on LIBLINEAR; Fan et al., 2008) with balanced class weights and $C = 0.5$, wrapped in `CalibratedClassifierCV` with three folds so that it outputs probabilities, logistic regression (balanced weights, $C = 1.0$) and Complement Naive Bayes (Rennie et al., 2003). A balanced logistic regression is the meta-learner; scikit-learn (Pedregosa et al., 2011) generates its training inputs with an internal three-fold split. The final stack is fitted on the full training partition.

4.4 Transformer models

We fine-tune `xlm-roberta-base` and `google/muril-base-cased` as sequence classifiers with the Transformers library (Wolf et al., 2020). Training uses AdamW (Loshchilov and Hutter, 2019) with learning rate 2×10^{-5}, weight decay 0.05, a cosine schedule with 10% warm-up, five epochs, batch size 8 with two gradient-accumulation steps, and a maximum length of 192 subword tokens. Mixed precision (Micikevicius et al., 2018) and gradient checkpointing (Chen et al., 2016) let each run fit on one 16 GB GPU. The model after the last epoch is kept; no checkpoint selection is done. All random seeds, including the fold split, are fixed to 42, and the runs used PyTorch 2.11 and Transformers 5.18. Each fold model predicts the development and test sets, and the five softmax outputs are averaged.

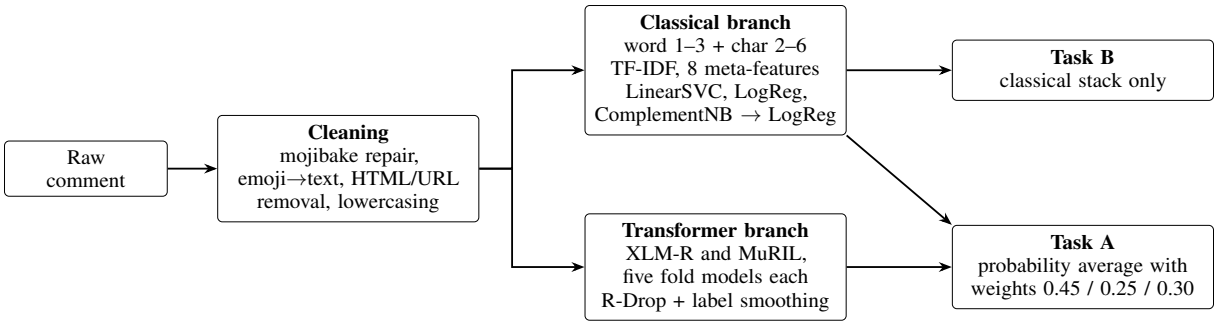


Figure 1: Overview of the system. The classical branch is used for both subtasks; the transformer branch contributes only to Task A.

The loss follows R-Drop. Each batch goes through the network twice with different dropout masks, giving distributions p_1 and p_2:

$$\mathcal{L} = \tfrac{1}{2}\big(\mathcal{L}^{\epsilon}_{\mathrm{CE}}(p_1, y) + \mathcal{L}^{\epsilon}_{\mathrm{CE}}(p_2, y)\big) + \alpha\, \mathcal{L}_{\mathrm{symKL}}(p_1, p_2), \quad (1)$$

where $\mathcal{L}^{\epsilon}_{\mathrm{CE}}$ is cross-entropy with label smoothing $\epsilon = 0.1$ and $\alpha = 0.5$. In Task B the cross-entropy also carries class weights computed from the training counts (Gender 0.39 up to Geo-political 2.83).

4.5 Combining the models

For Task A the final probability is

$$P = w_c P_{\mathrm{cls}} + w_x P_{\mathrm{xlmr}} + w_m P_{\mathrm{muril}}. \quad (2)$$

We searched all weight triples summing to one on a 0.05 grid and kept the one with the best development macro-F1, $(0.45, 0.25, 0.30)$. A threshold search on the NON-HATE probability (0.35 to 0.65, step 0.005) left the default of 0.5 unchanged. For Task B we blended the classical stack with XLM-R (classical weight 0.5 to 1.0); the best weight was 1.0, so the classical stack was submitted alone. MuRIL was not part of this blend because it produced no predictions for two classes.

5 Results and Discussion

5.1 Official results

Table 2 gives the submitted systems. Macro-F1 fell by about 0.03 on Task A and about 0.04 on Task B between development and test. Part of the Task A gap is expected, because the ensemble weights were picked on the development set. The ranking is also tight: on Task A, the 13 systems from rank 10 to 22 lie within 0.010 macro-F1 of each other, and on Task B, ranks 12 to 20 lie within about 0.011. The best Task B macro-F1 on the leaderboard was 0.7367, against our 0.6246.

Task	Dev F1	Test F1	Test Acc.	Rank
A	0.8288	0.7977	0.7978	18/30
B	0.6648	0.6246	0.6869	15/29

Table 2: Development and official test results of the submitted systems (Task A: three-model average; Task B: classical stack).

5.2 Task A

On Task A the three models are close to one another (Table 3): development macro-F1 is 0.800 to 0.806 and cross-validated macro-F1 is 0.791 to 0.813. Averaging them helps: adding XLM-R to the classical stack gives 0.8213 and adding MuRIL as well gives 0.8288, 0.022 above the best single model. The two encoders vary more from fold to fold than their means suggest: fold-level macro-F1 ranges from 0.771 to 0.804 for XLM-R and from 0.775 to 0.816 for MuRIL. At the default threshold, MuRIL leans towards the HATE class (recall 0.833, precision 0.785), while the classical stack is balanced (0.803 and 0.792). The models differ in what they see (n-gram strings against contextual subword representations), which is a plausible reason for the gain, although we did not analyse their disagreements. Since the weights were chosen on the same 806 comments that we report, the 0.8288 is an optimistic estimate, and the test score of 0.7977 is the better estimate.

5.3 Task B

On Task B the classical stack is clearly ahead, with development macro-F1 0.6648 against 0.5321 for XLM-R and 0.4944 for MuRIL, and the same order holds in cross-validation (Table 3). The per-class scores (Table 4) show where the encoders lose. MuRIL never predicted Others or Violence on the development set, and it favoured Gender (recall 0.924, precision 0.636). XLM-R predicted every

Task	Model	CV F1	Dev F1
A	Classical stack	0.8125	0.8002
A	XLM-R	0.7908	0.8002
A	MuRIL	0.7907	0.8064
A	Classical + XLM-R (0.5/0.5)	–	0.8213
A	All three (0.45/0.25/0.30)	–	0.8288
B	Classical stack	0.5953	0.6648
B	XLM-R	0.5151	0.5321
B	MuRIL	0.4684	0.4944

Table 3: Macro-F1 of each model on the out-of-fold training predictions (CV) and on the development set.

Class	Classical	XLM-R	MuRIL
Gender	0.824	0.707	0.753
Geo-political	0.720	0.526	0.667
Others	0.530	0.373	0.000
Political	0.809	0.720	0.828
Religion	0.756	0.605	0.719
Violence	0.351	0.262	0.000
Macro-F1	0.665	0.532	0.494
Accuracy	0.729	0.595	0.676

Table 4: Task B development F1 per class.

class but with low precision on Violence (0.196) and Others (0.413). The stack also finds Violence hard (F1 0.351), but it still has the highest score there.

We recomputed the development confusion matrix of the classical stack from the same code and data (it reproduces the 0.6648 above). It makes 107 errors on 395 comments, and 79 of them have Others or Violence as the true or the predicted class. The largest single confusions are Gender comments predicted as Others (17) and Others comments predicted as Gender (12); of the 28 Violence comments, 7 are labelled Others and 5 Gender. We did not examine why these classes are confused with one another. On the test set, the stack assigned 37 comments (9.3%) to Violence and 32 (8.1%) to Geo-political, against 7.0% and 5.9% of the training data, which is consistent with the balanced class weights pushing predictions towards the small classes.

The training logs do not point to overfitting. For XLM-R on one Task B fold, validation loss was still falling at epoch 5 and validation macro-F1 had risen from 0.085 after the first epoch to about 0.51. Five epochs at a fixed learning rate may simply be too little when four of the six classes have only 186 to 449 training examples; we did not test longer training, other learning rates, or oversampling, so we cannot say whether the encoders could be repaired. What we can say is that, under the same setup that worked on Task A, they did not learn the rare classes, and class weighting alone did not change that.

5.4 Interpretation: why sparse features may help

Character n-grams do not depend on a fixed spelling, so they can match the many romanized variants of a Kannada word, and balanced linear models are stable on a few thousand examples. We think both properties help here, but we ran no ablation (for example, without the character features), so this remains an interpretation rather than a measured result. For reference, one fold took about 25 minutes (XLM-R) or 21 minutes (MuRIL) on Task A and about 12 and 11 minutes on Task B on a Tesla T4.

6 Conclusion

We combined a classical stack over word and character TF-IDF features with R-Drop-regularized XLM-R and MuRIL for the HASTIKA shared task. Averaging all three models gave an official macro-F1 of 0.7977 on Task A. On Task B the encoders failed on the rare classes, and the classical stack alone reached 0.6246. For similar small, imbalanced code-mixed data, a sparse baseline deserves to be tried first, and an encoder should be added only where the development data show a gain. Natural next steps are longer or re-tuned transformer training, imbalance-aware objectives or sampling such as focal loss (Lin et al., 2017) or oversampling (Chawla et al., 2002), an ablation of the feature groups, and a closer look at the Others and Violence classes.

Limitations

The development set is small, especially for the minority classes of Task B (23 to 28 comments each), so single predictions move class-level F1 noticeably. Ensemble weights were tuned on that same set. We report one run per configuration and no repeated seeds. There is no ablation of the feature groups or of the cleaning steps. Three of the eight meta-features are almost inactive, as explained in Section 4.2. The abusive-word list is short and built mainly from English and romanized Hindi terms. Final models were trained on the training partition only, not on training plus development data. We did not carry out an error analysis on

the test set. We did not test longer training, focal loss, or oversampling for Task B, although these are standard remedies for rare classes. Our conclusion that encoders fail on Task B is therefore conditional on this fixed setup. Additionally, we did not transliterate or normalize Kannada-script text; the 45 Kannada-script comments in Task A were treated as-is.

Ethical Considerations

The system was built for a shared task and is not meant for automatic moderation. Wrongly flagged comments can silence legitimate speech, and missed ones leave abuse in place, so any real use would need human review. We worked only with the released data and made no attempt to identify authors.

References

Valerio Basile, Cristina Bosco, Elisabetta Fersini, Debora Nozza, Viviana Patti, Francisco Rangel, Paolo Rosso, and Manuela Sanguinetti. 2019. SemEval-2019 task 5: Multilingual detection of hate speech against immigrants and women in Twitter. In *Proceedings of the 13th International Workshop on Semantic Evaluation*, pages 54–63.

Aditya Bohra, Deepanshu Vijay, Vinay Singh, Syed Sarfaraz Akhtar, and Manish Shrivastava. 2018. A dataset of Hindi-English code-mixed social media text for hate speech detection. In *Proceedings of the Second Workshop on Computational Modeling of People's Opinions, Personality, and Emotions in Social Media*, pages 36–41.

Tommaso Caselli, Valerio Basile, Jelena Mitrović, and Michael Granitzer. 2021. HateBERT: Retraining BERT for abusive language detection in English. In *Proceedings of the 5th Workshop on Online Abuse and Harms*, pages 17–25.

Bharathi Raja Chakravarthi, Ruba Priyadharshini, Navya Jose, Anand Kumar M, Thomas Mandl, Prasanna Kumar Kumaresan, Rahul Ponnusamy, Hariharan R L, John P. McCrae, and Elizabeth Sherly. 2021. Findings of the shared task on offensive language identification in Tamil, Malayalam, and Kannada. In *Proceedings of the First Workshop on Speech and Language Technologies for Dravidian Languages*, pages 133–145.

Bharathi Raja Chakravarthi, Ruba Priyadharshini, Vigneshwaran Muralidaran, Navya Jose, Shardul Suryawanshi, Elizabeth Sherly, and John P. McCrae. 2022. DravidianCodeMix: Sentiment analysis and offensive language identification dataset for Dravidian languages in code-mixed text. *Language Resources and Evaluation*, 56(3):765–806.

Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. 2002. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357.

Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*.

Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. 2020. Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8440–8451.

Thomas Davidson, Dana Warmsley, Michael Macy, and Ingmar Weber. 2017. Automated hate speech detection and the problem of offensive language. In *Proceedings of the Eleventh International AAAI Conference on Web and Social Media*, pages 512–515.

Rong-En Fan, Kai-Wei Chang, Cho-Jui Hsieh, Xiang-Rui Wang, and Chih-Jen Lin. 2008. LIBLINEAR: A library for large linear classification. *Journal of Machine Learning Research*, 9:1871–1874.

Paula Fortuna and Sérgio Nunes. 2018. A survey on automatic detection of hate speech in text. *ACM Computing Surveys*, 51(4):1–30.

Adeep Hande, Ruba Priyadharshini, and Bharathi Raja Chakravarthi. 2020. KanCMD: Kannada CodeMixed dataset for sentiment analysis and offensive language detection. In *Proceedings of the Third Workshop on Computational Modeling of People's Opinions, Personality, and Emotion's in Social Media*, pages 54–63.

Divyanshu Kakwani, Anoop Kunchukuttan, Satish Golla, Gokul N.C., Avik Bhattacharyya, Mitesh M. Khapra, and Pratyush Kumar. 2020. IndicNLPSuite: Monolingual corpora, evaluation benchmarks and pre-trained multilingual language models for Indian languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4948–4961.

Sanjana Kavatagi and Rashmi Rachh. 2025. HASTIKA: Hate speech and target identification in Kannada–English code-mixed text. *Language Resources and Evaluation*, 59(3):2811–2856.

Simran Khanuja, Diksha Bansal, Sarvesh Mehtani, Savya Khosla, Atreyee Dey, Balaji Gopalan, Dilip Kumar Margam, Pooja Aggarwal, Rajiv Teja Nagipogu, Shachi Dave, Shruti Gupta, Subhash Chandra Bose Gali, Vish Subramanian, and Partha Talukdar. 2021. MuRIL: Multilingual representations for Indian languages. *arXiv preprint arXiv:2103.10730*.

Simran Khanuja, Sandipan Dandapat, Anirudh Srinivasan, Sunayana Sitaram, and Monojit Choudhury.

2020. GLUECoS: An evaluation benchmark for code-switched NLP. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 3575–3585.

Xiaobo Liang, Lijun Wu, Juntao Li, Yue Wang, Qi Meng, Tao Qin, Wei Chen, Min Zhang, and Tie-Yan Liu. 2021. R-Drop: Regularized dropout for neural networks. In *Advances in Neural Information Processing Systems*, volume 34, pages 10890–10905.

Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. 2017. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2980–2988.

Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *International Conference on Learning Representations*.

Binny Mathew, Punyajoy Saha, Seid Muhie Yimam, Chris Biemann, Pawan Goyal, and Animesh Mukherjee. 2021. HateXplain: A benchmark dataset for explainable hate speech detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 14867–14875.

Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. 2018. Mixed precision training. In *International Conference on Learning Representations*.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.

Telmo Pires, Eva Schlinger, and Dan Garrette. 2019. How multilingual is multilingual BERT? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4996–5001.

Jason D. M. Rennie, Lawrence Shih, Jaime Teevan, and David R. Karger. 2003. Tackling the poor assumptions of naive Bayes text classifiers. In *Proceedings of the Twentieth International Conference on Machine Learning*, pages 616–623.

Anna Schmidt and Michael Wiegand. 2017. A survey on hate speech detection using natural language processing. In *Proceedings of the Fifth International Workshop on Natural Language Processing for Social Media*, pages 1–10.

Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. 2016. Rethinking the Inception architecture for computer vision. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2818–2826.

Sida Wang and Christopher D. Manning. 2012. Baselines and bigrams: Simple, good sentiment and topic classification. In *Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics (Short Papers)*, pages 90–94.

Genta Indra Winata, Samuel Cahyawijaya, Zihan Liu, Zhaojiang Lin, Andrea Madotto, and Pascale Fung. 2021. Are multilingual models effective in code-switching? In *Proceedings of the Fifth Workshop on Computational Approaches to Linguistic Code-Switching*, pages 142–153.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, and 3 others. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45.

David H. Wolpert. 1992. Stacked generalization. *Neural Networks*, 5(2):241–259.

Marcos Zampieri, Shervin Malmasi, Preslav Nakov, Sara Rosenthal, Noura Farra, and Ritesh Kumar. 2019. Predicting the type and target of offensive posts in social media. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1415–1420.